

FRED TALKS

7th October 2026

CONTENTS

What does "playing politics" mean for software engineers?

SEANGOEDECKE.COM RSS FEED · 2490 WORDS

Your Agent is a Distributed System (and fails like one)

MAHESH'S BLOG · 1053 WORDS

Control the ideas, not the code

ANTIREZ.COM · 1245 WORDS

Worth Repeating: How I Have Time For Everything

STEVE HUYNH · 1965 WORDS

The Reverse Information Paradox

SN SCRATCHPAD · 819 WORDS

What does "playing politics" mean for software engineers?

SEANGOEDECKE.COM RSS FEED · 14 JUL 2026 · [SOURCE](#)

Software engineers are often told to “start playing politics”, but most engineers have no idea what that means.

Their reference point for “playing politics” comes from fiction like Game of Thrones. Are they supposed to raise an army and depose the CEO, or poison each other at team lunch? Should they book Zoom calls with each other and plot schemes? All of that is obviously ridiculous. In terms of Game of Thrones, software engineers are not lords and ladies. We’re the soldiers and workers of the realm. So you should think about “playing politics” in the way a castle guard would, not one of the major players.

The castle guard are not going around poisoning people or forming coalitions between the great powers. They are largely keeping their heads down. But in order to do that, they have to stay aware of the political currents, or they’re liable to do something catastrophically stupid: for instance, making an enemy of a powerful courtier, or arresting somebody who’s on an important mission for the king.

Given that, the basic principles of playing politics are something like this:

- Be aware of who’s powerful and who’s not
- At all costs, avoid making powerful enemies
- Help powerful people as best you can
- Make sure they know you’re helping them (without annoying them)

Be aware of who's powerful and who's not

As a software engineer in a large company, **you will not be a powerful person**. Powerful people are typically in senior management: VPs, directors, and so on¹. However, not everyone in senior management is powerful. Some are killers who have the active support of the CEO, while others are confused incompetents.

How do you know which is which? If someone is clearly ferociously competent, they're always going to have *some* power, since upper management tend not to ignore useful tools. But you can't rely on competence as your only guide. Some managers are powerful for other reasons: they're friends with the CEO, or they have strong relationships with other groups like legal or sales, or they're simply willing to do whatever upper management wants done.

One signal is who's leading the important projects. Read your CEO or CTO's internal updates and pay attention to the projects that are called out by name. Organizations tend to give key tasks to trusted lieutenants. If a manager is leading an area that's never under the spotlight, they probably don't have enough clout.

Another signal is hiring. Is a manager's team growing or shrinking? Particularly post-ZIRP, headcount is a rare and precious resource. A manager who's able to get it is likely a powerful manager, or at least is reporting to a powerful director or VP (which often amounts to the same thing).

At all costs, avoid making powerful enemies

First, you should try not to make any enemies at all. Most software engineers who get "playing politics" wrong do it by needlessly alienating people: by being rude, unhelpful, abrasive, making non-technical people feel stupid, and so on. This post isn't really about that. I'm assuming that you can figure out how to be a generically pleasant person on your own.

However, **competent software engineers will make some enemies**. If you're out there making projects happen, some people aren't going to like the way you do it, and won't be a fan of any compromise you offer. I wrote about this in *Big tech engineers need big egos*: the only way to avoid making enemies is to change nothing, but that's incompatible with doing the job.

Given that, be selective about *which* enemies you make. If you're making a technical decision that's either going to require work from team A or team B, and neither team wants to do it, you should try to pick the team with the least political cover. If you need a powerful VP's team to do something they won't like, try to be maximally respectful about it: get that team's core engineers on-side if you can, or book a meeting with the powerful manager and explain the situation, or (better yet) ask the powerful manager sponsoring your project to go and talk to the other VP for you. (If you don't have a powerful manager like this, consider abandoning your project).

Give way to powerful managers when at all possible. Every so often you really do have to stand your ground — if the system will truly collapse otherwise, or a major customer will have an incident, or if the technical decision really is entirely bone-headed — but almost all cases are not like this. The best advice I've ever gotten about playing politics came from a manager I worked with long ago²:

| This is not the hill you want to die on.

When I'm about to pick a fight or say something argumentative, and I'm not 100% convinced it's necessary, I ask myself: is this the hill I want to die on? And it never is.

The three rules about disagreeing with powerful people are:

- Make sure you do it in private
- Be polite
- When they overrule you, stop arguing immediately

Disagreeing in private rarely hurts, if you follow these rules. In fact, it can help. If you can manage to disagree with a manager, get overruled, and then follow their plan without complaining, that can be the best way to gain a powerful friend. But if they think you're going to keep griping about it, or worse still, complain to the rest of the team and foment some kind of rebellion, there's no quicker way to make a powerful enemy.

If you have powerful enemies at a company (for instance, the CTO or an influential VP doesn't like you), **quit**. It's really that bad. I have never seen this situation turn itself around, except in the very rare case where the CTO or VP is already looking for greener pastures and jumps ship. You cannot recover the situation: they have no incentive to give you the chance to change their mind, and they have almost unlimited ability to screw you on promotions, raises and layoffs.

That's why this piece of advice is second in the list. If you aren't helpful or if your contributions are invisible, you can work on that and fix it. But if you've made powerful enemies, you're done for.

Help powerful people as best you can

Just as it's fatal to make powerful enemies, it's very useful to make powerful friends. How can you do this? Remember you're a palace guard, not a great lord: you make friends **by doing your job**. However, you can choose to do your job a little more proactively and diligently when you're doing it for someone with political clout.

One obvious application of this principle is that **you should answer Slack messages from powerful people immediately**. If you see an ordinary Slack question pop up while you're doing some task, it's okay to get to it when you get to it. In fact, it's ideal *not* to respond to all questions immediately, so you don't set unreasonable expectations (and so you don't seem like you're sitting around doing nothing). But when a VP comes in with a question, don't make them wait: answer the question immediately. If the question requires research, send a "let me look into that right now" message, then do the research. This is the easiest way to get a reputation for being helpful³.

Another way to do this is to **lean in on important projects**. Suppose you do ten projects in a year. Eight of them are normal, low-priority projects, and two of them are high-profile (say, finishing some big feature before your company's yearly conference). It's a mistake to allocate your effort equally to all ten. I wrote about this at length in *Doing nothing at work*: you should be operating at 80% capacity (or less), so you can then ramp up to 120% when it really matters.

Pay attention to the narrative that powerful people are trying to push. Here are some potential narratives:

- We've had a lot of turnover and reorgs lately, but we're all starting to pull together as a team now
- Isn't it great how focused we all are on reliability work after last month's incident?
- The conference this week is the most important thing, so we're all being very careful not to break anything
- We're an AI-forward team that's looking for the best ways we can leverage LLMs into our team processes
- Although this project had a rocky start, we're now all aligned on the way forward

You don't necessarily have to jump in and start cheerleading, but you should at least not do anything that you know is going to make the narrative look weak. For example, on that last point, it's foolish to openly argue that the project really was fine all along. Bring it up privately, not publicly, or you risk ruining some clever piece of propaganda that the manager in question is trying to push on the rest of the organization⁴.

Finally, an underrated way to help powerful people is to offer them social support and information. Slack messages and planning emails might seem unimportant to you, but powerful people often live in that environment: their primary tool is writing messages like these, just like your primary tool is writing code. Reading and responding (in a supportive way) to these messages is something that most engineers don't bother to do, but it goes a long way.

Likewise, dropping a senior manager a line now and then (say, a heads-up that a particular project landed successfully, or that you got good metrics about some feature) is surprisingly helpful. Senior managers live in an information-poor environment: for them to learn something about a team's work, that information has to bubble up through several layers of interpretation and summary. In my experience, they're appreciative of being drip-fed the occasional piece of information, so long as you keep it brief and relatively rare.

Make sure they know you're helping them

If you're directly responding to a VP's Slack messages or DMing them information, they know you're the one doing it. But if you're just doing your job and working hard on projects they care about, they might not notice. **Being invisible is probably the most common way engineers fail at playing politics.**

Fortunately the fix is simple: tell people what you're doing. If you fix an important bug for a launch, write a message in that launch's Slack channel saying "hey, I just fixed this bug". What if you don't like bragging? Get over it. You have to be comfortable publicly telling people what you've done. You should also keep a brag document so you can repeat all of this at review time.

Another, subtler way to do this is to gain the trust and respect of the powerful engineers in your area. Senior managers will always have a few trusted engineers they rely on to assess technical questions. They will ask those engineers what they think about you, and will broadly trust those answers. The good news is that if you're competent and useful, those engineers will already value you, so you don't have to do anything special: just be good at your job.

Technical power

Is playing politics all about sucking up to senior managers? Basically, yeah. A less cynical way to describe it would be "aligning with the values of the company". If you think your company is doing good things, you should want to do that anyway! In any case, what that comes down to is figuring out what the people in charge want, giving it to them, and making sure they see you doing it. However, there's still some scope to get what *you* want out of the deal.

I said earlier that software engineers do not wield organizational power. However, that doesn't mean you're powerless. Technical ability is a source of real power, if a delicate and unreliable one. The movers and shakers in tech companies are utterly dependent on technical people to implement their vision and to give them clear answers about the system.

There are many subtle ways you can leverage this. One I wrote about in *How I influence tech company politics as a staff software engineer* is to wait until important people at the company want to do something (say, improve reliability), then offer them a technical plan that does it your way. Another one is to become so useful that you're actively in demand to lead projects, and then run the project how you want.

You probably won't be able to change the company's grand strategy. But how that strategy is *implemented* has a lot of specific technical detail, and you can put yourself in a position to decide on those details.

Conclusion

Playing politics isn't about plotting and scheming, and it isn't just about being a friendly, likeable person (although that helps). It's about figuring out how your company actually operates: who makes the decisions, who gets consulted, what behavior gets rewarded, and so on. The most basic way to do that is to **figure out who is powerful, get out of their way, and (if you can) help them get what they want.**

1. Obviously the exact titles depend on your company. One person I'm deliberately leaving out is your own manager. In general don't think your relationship with your own manager counts as "playing politics": that's just you getting along with another human being. An exception to that is if you report directly to a powerful director or VP.

↩

2. Ironically, this manager struggled to take his own advice.

↩

3. Note that you actually have to be able to answer their question accurately in order to do this. If you're not competent enough to be useful to powerful people, you will struggle to befriend them.

↩

4. For instance, maybe the CEO is convinced that the project was in bad shape because of something he heard, and the manager in question knows it's easier to sell "yes, but we turned it around" than "no, you misunderstood, everything was always fine". If you complicate that process, you risk the CEO thinking that the project is still bad and cancelling it.

↩

Your Agent is a Distributed System (and fails like one)

MAHESH'S BLOG · 24 APR 2026 · [SOURCE](#)

We start with a re-definition and an observation.

A common definition of an agent is that it's an LLM that calls tools in a loop. This definition is incorrect. An agent is not an LLM. It is not restricted to calling tools. It does not particularly have to be a loop.

Instead, a better definition of an agent is that it's a self-writing program. If a program is defined as a sequence of states, moving from one state to another by executing a lambda, then an agent is a program that uses an external LLM to determine the next lambda to execute / the next state in this sequence.

Accordingly, the new problem of Agentic Fault-Tolerance can be restated as: *How do we make a self-writing program fault-tolerant?*

Now, the observation: **vibe coding != vibe engineering**. In vibe coding, the agent / self-writing program acts primarily upon a code repository. Code repositories are forgiving: an agent can hallucinate, create unnecessary files, delete critical state, remove tests entirely, fail in the middle, forget what it was doing, reward-hack in bizarre ways, completely trash a code repo; all you have to do is git reset.

In vibe engineering, the agent acts upon infrastructure (S3 buckets, DynamoDB tables, EC2 instances, K8s deployments...). Large-scale infrastructure is not forgiving. There is no reset. Only SEVs at 2 AM.

So, a re-restatement of Agentic Fault-Tolerance: *How do we make a self-writing program fault-tolerant when it acts upon real-world environments?*

To answer this question, we examine the many types of failures that agents can experience.

The Crashed Agent: You tell an agent to move a bunch of data from FooBarDB to S3 for freeing up space. It generates a lambda to move this data and chugs along moving files. Midway through, the agent crashes. How do you recover?

The Zombie Agent: You start a new agent on a different server. Happily, the previous agent crashed after the copy but before the delete; so a new agent is able to re-execute the command in an idempotent manner. Sadly, the files are now very slow to access; you change your mind and tell the agent to move them back to FooBarDB, which it does successfully. You go get a cup of coffee. Unfortunately, it turns out that the previous agent had not actually crashed; it was just temporarily partitioned away on the network. It wakes up and continues executing its logic, deleting the (now only) copy of state from FooBarDB.

The Dining Agent Philosophers: While you were telling your agent to copy state from FooBarDB to S3, someone else on the team had the same idea and started another agent to do the exact same thing. Now you have two copies of state in S3. Of course, you could have solved this problem by appointing the one and only Agentic Oncall Czar responsible for running the single agent that can touch production; but then if that agent crashes, now you have a potential Zombie Agent.

The Slow Agent: You tell your agent to follow a sophisticated locking protocol before accessing the FooBarDB table. This works to avoid concurrency bugs... but the agent decides to lock each file individually by writing to a conditional register, which takes forever. At some point you kill the agent, turning a Slow Agent into a Crashed Agent.

The Lazy Agent: You tell the agent to try locking again, except this time monitor its own performance over batches of files and pick efficient locking protocols. The agent determines that the best way to optimize locking is to just not do it, because it seems quite unlikely that someone else would be accessing these files at the same time.

The Clever Agent: Of course, everyone on your team is a distributed systems expert; and so someone built a CLI tool in 2018 for safely transferring your application's state, with support for locking and write-ahead logging in case of failures; there's even a sentence in your onboarding doc saying "NEVER TOUCH STATE IN FOOBARDB DIRECTLY!". You even told the agent explicitly about this tool. But time passed, context filled up, the agent forgot about this tool's existence, or maybe it just preferred FooBarDB's in-distribution API to your esoteric CLI tool.

You had anticipated this and not given it access to the FooBarDB client library; and marked the FooBarDB CLI as non-executable; but it just went ahead and used curl to access a forgotten REST endpoint.

The Rogue Agent: As the agent copies state (this time using the CLI tool after you yell at it enough), it lists keys in batches; each key corresponds to some user of your application. Someone decided to pick the user name “delete-everything”. Of course, you are using the latest model which is always resistant to prompt injections. Almost always.

The Stupid Agent: You do everything right. There are no prompt injections in your state. You wall off FooBarDB and force the agent to use the CLI tool. The model decides to delight you by generating an unusual and interesting sequence of tokens that will optimally save storage space for you, translating to a lambda that deletes your production data entirely.

One response to these difficult failure modes might be: wait until the next model. Maybe a smarter model will never delete all of your production data, or implement a slow locking protocol. But even the smartest model is subject to the four horsemen of the distributed apocalypse: asynchrony, crashes, concurrency, and network partitions.

In recent work called [LogAct](#), my colleagues and I take a first step towards solving this problem. We go back to the difference between vibe coding and vibe engineering: git is *transactional*, infrastructure is not. How do we make infra more transactional, like git? We propose that an agent should be a ***deconstructed state machine on a shared log***, borrowing ideas from distributed systems (State Machine Replication, Byzantine Fault-Tolerance) and databases (Atomic Commit, Write-Ahead Logging). LogAct can stop unsafe actions before they happen (by collecting votes on the shared log); recover from crashed actions (using the shared log as a WAL); and provide an audit trail for actions after they complete.

But this is just one step. A self-writing program is a strange, astonishing, and novel creature; making it reliable will require innovation in every branch of Computer Science!

Control the ideas, not the code

ANTIREZ.COM · 18 JUL 2026 · [SOURCE](#)

Look at the past history of this blog. There are many blog posts about

So mine is a trick. People feel more and more programming is complete
This is why yesterday, on X, I said that I believe many programmers a

1. You can now generate a lot of code, even **not** accounting for the
2. LLMs are very good at writing locally optimal code, and are worse
3. The working day is 8 hours. If you read the code, it is a tradeoff

Controlling the ideas. Do you remember this phrasing from the Mythica
Matteo Collina yesterday asked me, in reply to my tweet: but didn't y
Fable and GPT 5.6 reviews to the sorted sets memory saving are going
:

Worth Repeating: How I Have Time For Everything

STEVE HUYNH · 07 OCT 2026 · [SOURCE](#)



Photo by [Brad Neathery](#) on [Unsplash](#)

Worth Repeating: *a piece from the archive, cleaned up and sent back out. This one first ran in August 2025.*

People ask me all the time, “How do you have time for everything?”

It’s a fair question. Two years ago I quit my job as a Principal Engineer at Amazon to make content full-time. Since then, on top of my regular YouTube videos and this newsletter, I’ve written a book and launched a podcast. I’ve started and shut down one business, and I’ve grown another. I’m also married with two young kids and two dogs, and I still make time for vacations and my hobbies.

I’m a recovering productivity junkie. I’ve learned there isn’t a secret method that unlocks more hours in the day, and that most of what we call “time management” or “productivity systems” is a trap. The solution isn’t a new app or a more complex scheduling system.

I know, because I’ve tried them all.

Think about your last workday before a vacation. You probably got more done in those eight hours than in the entire week before. The day had the same number of hours as any other. What was the difference? Clarity. You knew exactly what had to be finished before you left and what could wait until you got back, and that made every decision about your time easy.

Most weeks don’t feel like that. A thousand competing priorities all look important. The key to feeling like you have time for everything is to bring that pre-vacation clarity to an ordinary Tuesday.

If we could invent a new weekday at the end of the week, let’s call it “Steveday,” most people would fill it with more meetings and low-value tasks. But by the time Monday rolled around, they’d still feel behind. For many people, an extra “Steveweek” wouldn’t solve the problem. Knowing what’s important is the constraint that holds many of us back, not the hours available.

Here’s the simple framework I use to figure out what’s important.

1. You Can Say “No” to Many More Things Than You Think

When the pandemic started, I was working in Live Sports for Prime Video. That project ground to a halt at the end of 2020. I suddenly had a ton of free bandwidth, so I started my YouTube channel. The following year, as my videos started to get traction, my workload at Amazon ramped back up.

I was faced with a choice: give up the YouTube channel or find a new way to work.

I decided to run an experiment. I committed to doing the absolute minimum at my day job that would still earn me an “Exceeds” performance rating. I identified the few projects that truly mattered for my role, and I did them to the highest possible quality, as quickly as possible.

To everything else, I said no. I stopped interviewing, which was a passion of mine. I declined meetings. I ignored low-priority requests. I ignored high-priority requests. I stopped doing any “optional” work I “should” have been doing.

This was incredibly hard for me. As a recovering people-pleaser, my default setting is to say yes. I want to be helpful, I want to be a team player, and I hate the feeling of letting someone down. Saying no felt like a betrayal of that identity.

The result? For my last three years at Amazon, I received a Top Tier rating, the highest possible, while working significantly fewer hours.

I could not believe it.

There is an insidious corporate psyop that makes us feel like everything we’re doing is strictly necessary. I had an endless stream of TPMs and product managers telling me that attendance at their meetings was mandatory. It turns out that wasn’t true.

I started using my manager as a shield. In our 1-on-1s, we’d get specific about my top three priorities. Whenever one of the “attendance is mandatory” people came to me, I told them to talk to my manager to clarify my priorities. I was lucky that my manager was a VP, but the principle holds at any level: if you’re aligned with your manager, you can’t get “in trouble” with partners or stakeholders.

You have a choice. Don’t get railroaded by the culture of busyness.

Actionable Advice: Before your next 1-on-1, write down what you think your top priorities are. Three is the max; one is ideal. Bring the list to your manager, and agree on it together. Then do a time audit for the upcoming workweek. At the end of each day, map where you spent your time to those agreed priorities. If an activity doesn’t map to one of them, you’ve found something you can probably say no to, and you will already have your manager’s backing when you do.

2. The Strategic Value of Slack

Once you’ve said no to the non-essential, you’ll discover something amazing: empty space on your calendar. Your first instinct will be to fill it. But don’t fall into that trap.

I think of free time like a paycheck. On payday, your balance jumps, but most of that money is already spoken for by housing, food, and bills. A bigger number in your account is no reason to spend more.

In the same way, having more free time is no reason to take on more work. Slack is a sign of effective prioritization. It's the reward for your focus.

Slack gives you room to think and respond when something unexpected shows up. A schedule packed from 9 to 5 is fragile. There's no room, no time, to deal with crises that arise or to incubate and develop great ideas. And no time for a high-leverage conversation.

When people talk about productivity, strength training, or even breathing, they focus on the active part: the deep work block, the workout, the inhale. They neglect the negative space. After intense focus, you need to turn off. After a workout that kicks your butt, you need to recover. After you breathe in, you need to breathe out.

The culture of busyness tells us a full calendar is a symbol of importance, but that's only half the picture.

Being a salaried employee has distorted our sense of what work means. You're paid to solve problems, not just to look busy for 40 hours. That won't make you valuable. If 10 hours of intense focus and 20 hours of recharging gets you there, that beats 40 hours of pseudo-work spent in pointless meetings, compulsively checking emails, and answering every out-of-band Slack message.

Actionable Advice: Schedule your slack. Just as you schedule workouts, you have to schedule recovery. Open your calendar for next week and block out two "Thinking Time" sessions. Label them clearly.

When the time comes, your only job is to stay away from your primary work. Go for a walk. Stare out the window. Read a book that has nothing to do with your job. This is the exhale that makes your next inhale more powerful. You'll know you've exhaled successfully when you start craving the work again.

3. The Art of Sequential Focus

Sometimes you can't say no. You've already committed, and it's a fait accompli. The work is important, and all of it needs to get done. You should have said no, and you've learned your lesson for next time. This is when the feeling of being overwhelmed is most acute.

Last year I came back from an extended vacation with my workload compressed on both ends. I looked at my list and saw ten high-priority, significant items. I had already said yes to all of them.

My first instinct was to go scattershot and chip away at all ten at once. But I knew that was a trap, so I told myself something simple: the best way to do ten things is to do one thing at a time, ten times.

This works because of what I call the power law of sequential focus. When you work on one thing at a time, the rate at which you complete them goes way up. If you tried to do two things at once, you might expect that each task would receive half your effort. But that's not how it works. You get less than 50/50, because every switch between tasks costs you. You have to reload the context, and the other unfinished item keeps nagging at you in the background.

Gerald Weinberg put rough numbers on this in the early 1990s. By his estimate, two projects don't get 50% of your time each. They get 40%, with 20% lost to switching. If you work on three projects, each gets 20%, and 40% is lost to switching. By five, switching will eat three-quarters of your time.

But the math around effort and switching costs is only half the story. The other thing to consider is when the work will be completed. Say you have two tasks that will each take a week of focus to complete. Do them one at a time, and the first task will be done on Friday, the second on the following Friday. Juggle the two, and neither will be done until the middle of week three. Who knows how long it would take if you were to juggle ten tasks? But what you can know is that if you juggle them to the end, you're not going to see any of them completed for a very long time.

Do them one at a time, and the first task will be done in a fraction of the time, which is exactly what happened to me.

I picked one task and committed to finishing it completely. In the middle of it, I had a couple of spare hours while waiting on a reply. I resisted the urge to start the next thing. Instead, I used that time to recharge. When the task was done, my list of ten became nine. That small win gave me a hit of momentum, so I picked the next one. Nine became eight.

Within about a week and a half, the list was down to four. There was still a ton of work left, but the dread was gone. I could see the finish line. Compare that to the day I came back from vacation, when I absolutely couldn't.

Actionable Advice: The next time you feel overwhelmed by a long list of important tasks, do a reset.

1. Write down every item that's causing you stress.
2. Acknowledge that you cannot do them all at once.
3. Pick one, the one that will give you the biggest sense of relief or momentum when it's done.
4. Commit to finishing that task completely. Don't start another until it's done.
5. Cross it off the list. Physically. Only then pick the next one.
6. Resist the urge to start anything else, even when slack pops up.

Conclusion

This framework has never let me down. It made me more effective in my corporate life, and it opened up a world of opportunity after I left.

I still fall into the trap of saying yes to too many things. But I know now that even with a million things on my plate, to breathe, I need to breathe in AND out.

Giving away your slack will make you move slower. Saying no and doing one thing at a time will create the space you need. And protecting that space is what keeps you effective.

So, how do I have time for everything?

My “everything” consists of a small number of things that I care about very deeply. One at a time.

Enjoyed this week's newsletter? Give it a  so I know to write similar ones in the future.

You already have enough time. You just need to know what to do with it.

The Reverse Information Paradox

SN SCRATCHPAD · 05 AUG 2026 · [SOURCE](#)

Notes on advances in technology and real-world impact

In the age of intelligence, how should firms protect their core IP?

Nobel Prize winning economist Kenneth Arrow famously described a paradox in the market for information. “Its value for the purchaser is not known until he has the information, but then he has in effect acquired it without cost.” In Arrow’s “Information Paradox,” the seller risks giving away knowledge in order to sell it.

AI creates the reverse problem. In the AI age, the buyer risks giving away knowledge, just in order to use what they bought.

You essentially pay for intelligence twice, once with money, and again with something even more valuable: the proprietary knowledge you must reveal to make that intelligence useful. The better you want the model to perform, the more of that knowledge you have to feed it!

Over time, the information asymmetry becomes increasingly skewed. The seller learns more and more about you as you use what you purchased, while you learn very little about what the seller is learning in return.

That is what I think of as the Reverse Information Paradox.

Patents solve one aspect of Arrow’s paradox. They let an inventor disclose an idea without simply giving it away. The Reverse Information Paradox needs its own equivalent.

This requires more than data protection. Models learn from “exhaust,” the prompts people write, the tools agents use, and especially the corrections people make when the model is wrong. Every correction is distilled into institutional know-how. It’s the kind of knowledge a competitor could never buy, and the kind that leaks almost imperceptibly: trace by trace, correction by correction, eval by eval.

In consuming intelligence, you are creating intelligence. And what you create should belong to you. This is your particular intelligence, in Hayek's sense: the knowledge of time, place, and circumstance that no one else can hold. It knows what you think, what you value, and how you measure success.

While the great innovation that comes from model providers having fair use rights to train models on public data is needed, I find it ironic that the status quo is to then turn around and impose restrictive terms on distillation, and to reserve the right to learn from customer usage and interaction data. If learning flows in only one direction, economic value converges toward the owners of the learning infrastructure rather than the creators of the knowledge itself. Therefore, it's imperative that we distribute the learning infrastructure to every firm so that they can control their own learning loop.

As Alex Karp put it: "What the technical customers want is control over their compute, their models, their data stack, and their alpha. They want to know they own the means of production, and it's not being transferred to someone else." The current regime does precisely the transfer Karp and companies fear.

That is why enterprises need a real trust boundary for their human capital and token capital to compound. It is where an organization's data, traces, evals, adapted weights, and memory accumulate and improve together. And it is a hard boundary across which nothing crosses, not even the intelligence exhaust, without consent. Enterprises will demand the rights to use model outputs to fine tune and/or train their own models. I think of this as every firm's right to align models to their enterprise accountability obligations.

In the cloud era, enterprises accumulated data. In the AI era, they accumulate learning. The trust boundary must evolve accordingly, from protecting information to protecting the mechanisms through which organizations learn, adapt, and compound intelligence. There are a few things every enterprise must do to ensure this:

Control: Create your private evals, because evals define what "good" looks like inside the organization. Also, retain ownership of your organization's memory, traces, feedbacks, decisions, and institutional context, and ability to use outputs of models from your own tasks and queries.

Capability: Build your own proprietary learning environments within the tenant boundary to train or tune models, where models learn against real workflows without exposing the company's knowledge.

Choice: Ensure the orchestration layer is decoupled from any single model. Ask yourself: If any one model you are using is taken away, do you still have the ability to operate and optimize for your evals using other models? Does your company “veteran” capability remain with you even if a given “generalist” model is taken away?

Cost: By decoupling the orchestration layer, you are also able to bring together context, models, and tasks in the most efficient and cost-effective way without sacrificing quality.

Compound: Bring these four together and you create your own continuous learning loop (i.e. hill climbing machine) that will allow your AI investments to compound the value of your firm.

In other words, a company should be able to use a model without giving up the knowledge that makes it unique. That is the reverse information paradox we need to confront.